

## Removal and insertion

We cannot update the existing tuple, but we can create new tuple with it; it will be copied into a new address:

```
tuple1 = ('one', 'two', 'three')
print(id(tuple1), tuple1)
print()

tuple2 = ('1', '2', '3')
print(id(tuple2), tuple2)
print()

tuple3 = tuple1 + tuple2
print(id(tuple3), tuple3)
print()
```

The output is:

```
140509667635968 ('one', 'two', 'three')

140509666902208 ('1', '2', '3')

14050966693664 ('one', 'two', 'three', '1', '2', '3')
```

## Sort

As tuples are immutable, we cannot implicitly sort them. But we can make use of the sort function to do so.

 **Note:** It will return as list.

```
sorted_tuple1 = sorted(tuple1)
print(id(sorted_tuple1), type(sorted_tuple1), sorted_tuple1)
```

The output is:

```
140509667589312 <class 'list'> ['one', 'three', 'two']
```

## Named tuple

The named tuple and normal tuple use exactly the same amount of memory because the field names are stored in the class. So we can either use tuple or named tuple. However, named tuple will increase the readability of the program. That's a bonus!

```
from collections import namedtuple
City = namedtuple("City", "name country status")
chennai = City("Chennai", "India", "Red")
print(chennai)
print("Size of named tuple: ", sys.getsizeof(chennai))

normaltuple = ("Chennai", "India", "Red")
```

```
print(normaltuple)
print("Size of normaltuple tuple: ", sys.
getsizeof(normaltuple))
```

The output is:

```
City(name='Chennai', country='India', status='Red')
Size of named tuple: 64
('Chennai', 'India', 'Red')
Size of normaltuple tuple: 64
```

The performance is:

- Index - O(1)
- Slice - O(k)
- in Operator - O(n)

n = number of elements  
k = k'th index  
1 = order of 1

 **Note:** As tuples are immutable they have only limited features.

To sum up, we should use lists when the collection needs to be changed constantly. We should use tuples when:

- Working with arguments
- Returning two or more items from a function
- Iterating over a dictionary's key-value pairs
- Using string formatting

Lists are complex to implement, while tuples save memory and time (a list uses 3000+ lines of code while tuple needs only 1000+ lines of C code). **END** 

## References

- [1] <https://www.laurentluce.com/posts/python-list-implementation/>
- [2] <https://github.com/python/cpython/blob/master/Objects/listobject.c>
- [3] Fluent Python, 2nd Edition [Book] from O'Reilly

## By: Thangaselvi Arichandrapandian

The author works in a leading bank as an AVP. He is an all-time learner influenced by the quote:

*"The more I learn, the more I realise how much I don't know."*  
— Albert Einstein.